Homework 0-7 · Code Grading with LLM-as-a-Judge

代码仓库：https://github.com/wangz61/code-grading-llm-judge

流程：Label Studio 标注参考答案 → 与模型输出合并为 JSONL → OpenAI Evals（model-graded，gpt-4.1 作评委）按 1–5 分评分 → 重复运行 5 次，计算平均分、方差与不确定性，并用单元测试实际运行代码做对照。

1. 需求对照

作业要求	实现
Label hw0-07_inputs.csv using Label Studio	pip install label-studio，本地启动；新建项目并导入 5 道题（图 1–5）
Define labeling interface	labelstudio/label_config.xml：显示题目；ideal 多行文本框填写参考答案（必填）；difficulty 单选 easy/medium/hard
Export the file as json file	Label Studio 导出 JSON（labelstudio/export.json，图 6）
Read the output of existing model	convert.py 读取 hw0-07_inputs_model_output.csv（含多行代码单元格）
Create the JSONL by merging the two files	按任务顺序合并；校验条数、题目文本与函数名一致；输出 data/samples.jsonl
Format: input / ideal / sampled	每行恰好这三个键（见 3 节）
Design a prompt with the scoring system 5–1	evals_registry/modelgraded/code_grading.yaml：rubric 5 fully correct … 1 incorrect，先推理再给分（cot_classify）
Create an eval config and run the evaluation	evals_registry/evals/code-grading.yaml；oaieval gpt-4.1-chat code-grading（图 7）
Interpret results (mean, variance, uncertainty)	analyze.py：5 次运行，平均分、题间方差、评委方差、标准误、t 与 bootstrap 95% 置信区间（图 8、5 节）

2. Label Studio 标注

标注界面定义（labelstudio/label_config.xml）：

```
<View>
  <Style>
    .task { font-size: 16px; font-weight: 600; padding: 8px 12px; background: #f4f6fa; border-radius: 6px; }
    .lsf-textarea, textarea { font-family: Consolas, monospace; }
  </Style>
  <Header value="Programming task (input)"/>
  <View className="task">
    <Text name="input" value="$input"/>
  </View>
  <Header value="Reference solution (ideal): a correct, idiomatic Python implementation"/>
  <TextArea name="ideal" toName="input" rows="8" editable="true" maxSubmissions="1" required="true"
```

```
           placeholder="def ..." showSubmitButton="false"/>
  <Header value="Difficulty"/>
  <Choices name="difficulty" toName="input" choice="single" showInline="true" required="true">
    <Choice value="easy"/>
    <Choice value="medium"/>
    <Choice value="hard"/>
  </Choices>
</View>
```

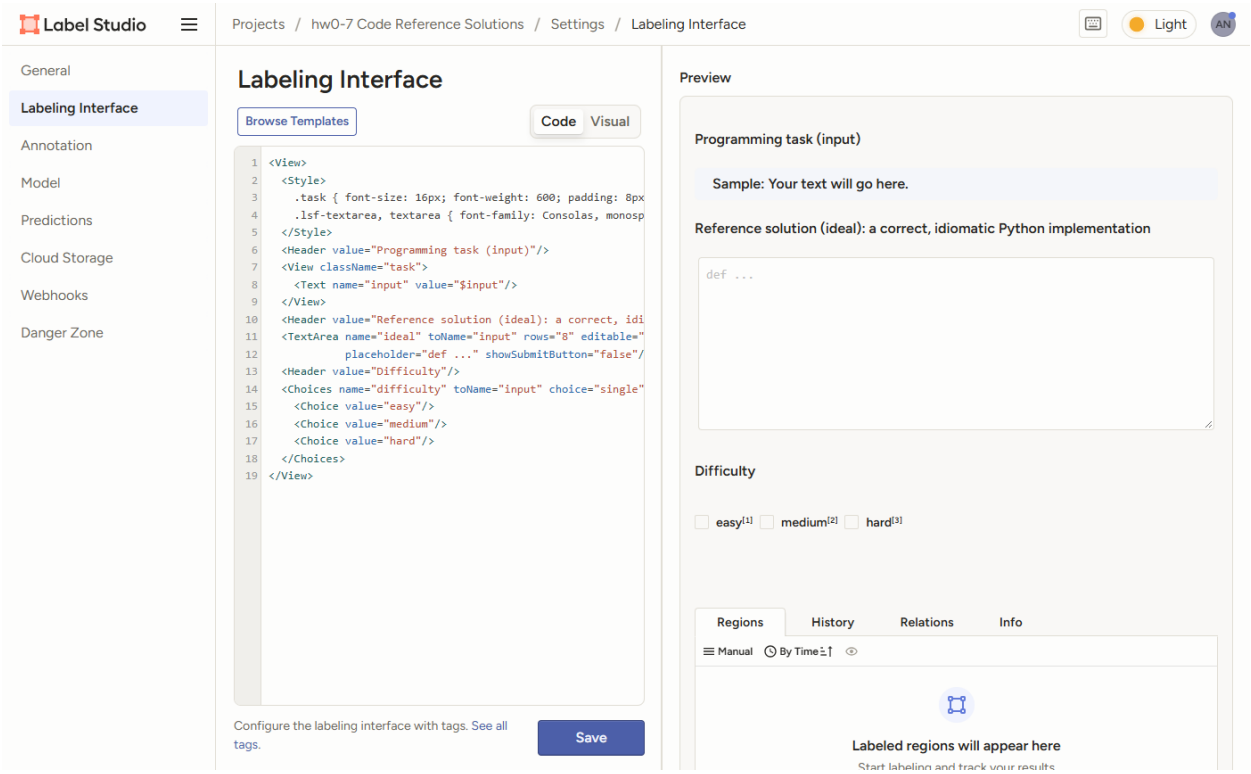


图 1 项目设置中的标注界面（Code 视图 + 预览）

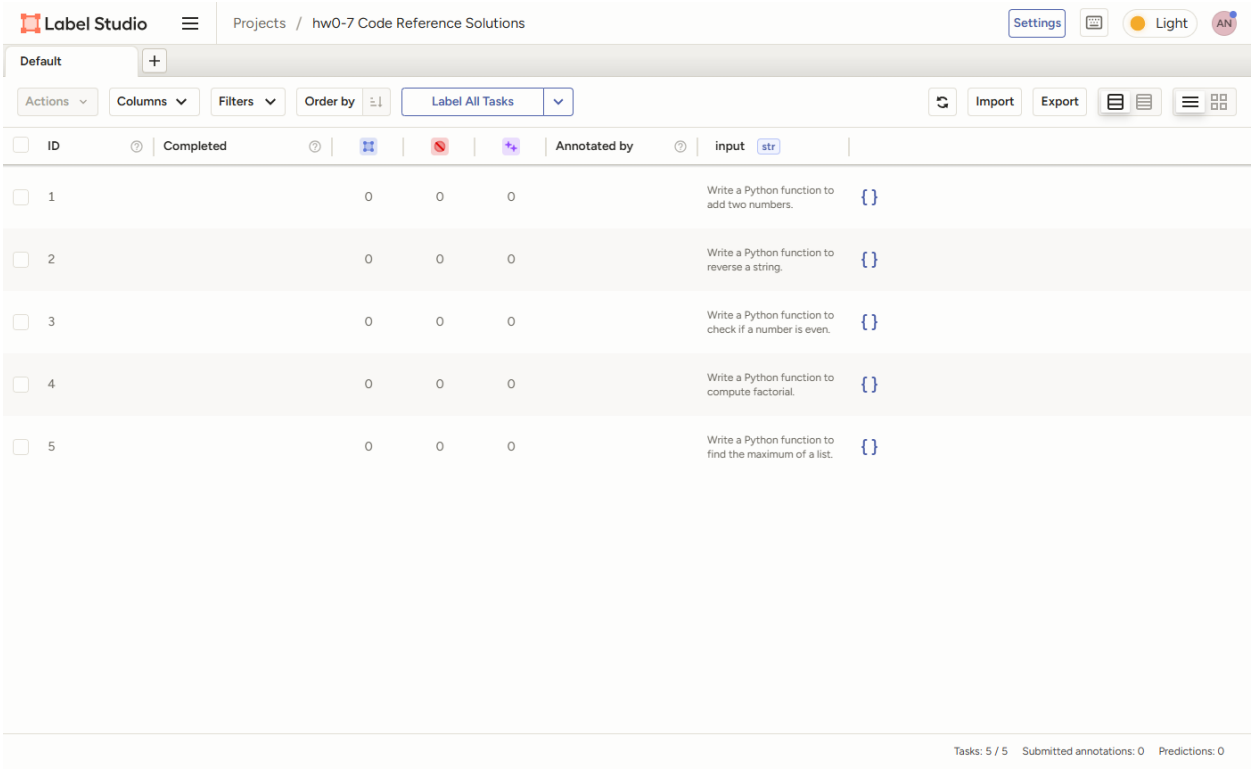


图 2 导入 hw0-07_inputs.csv 后的 5 个任务

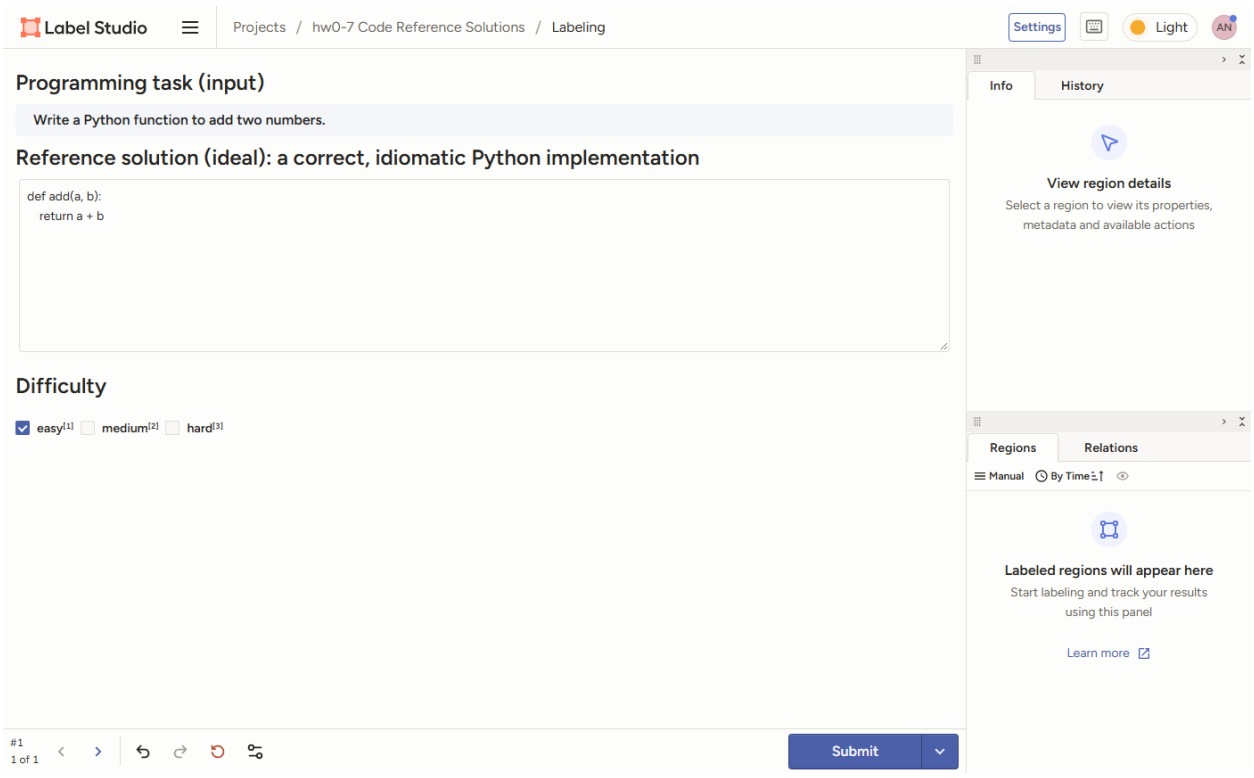


图 3 标注第 1 题：填写参考答案、选择难度，然后 Submit

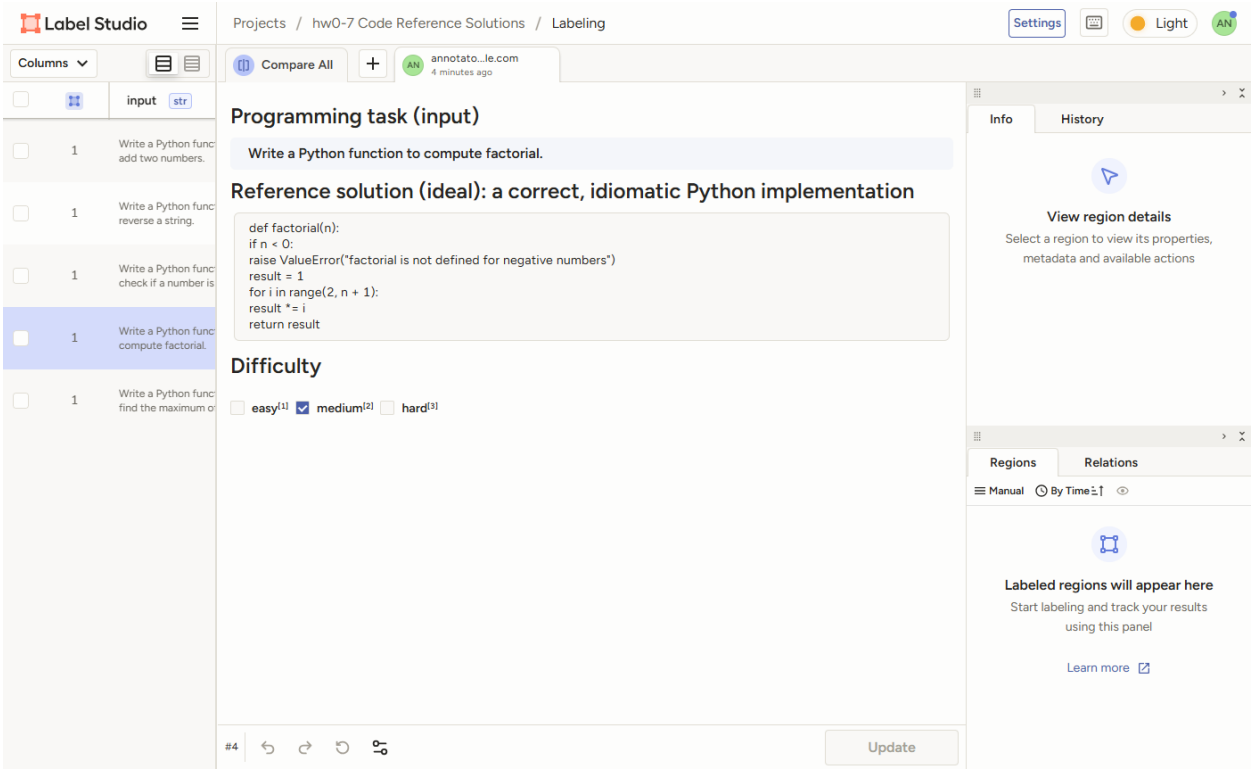


图 4 已提交的 factorial 标注（界面显示时不保留缩进，导出的 JSON 中保留）

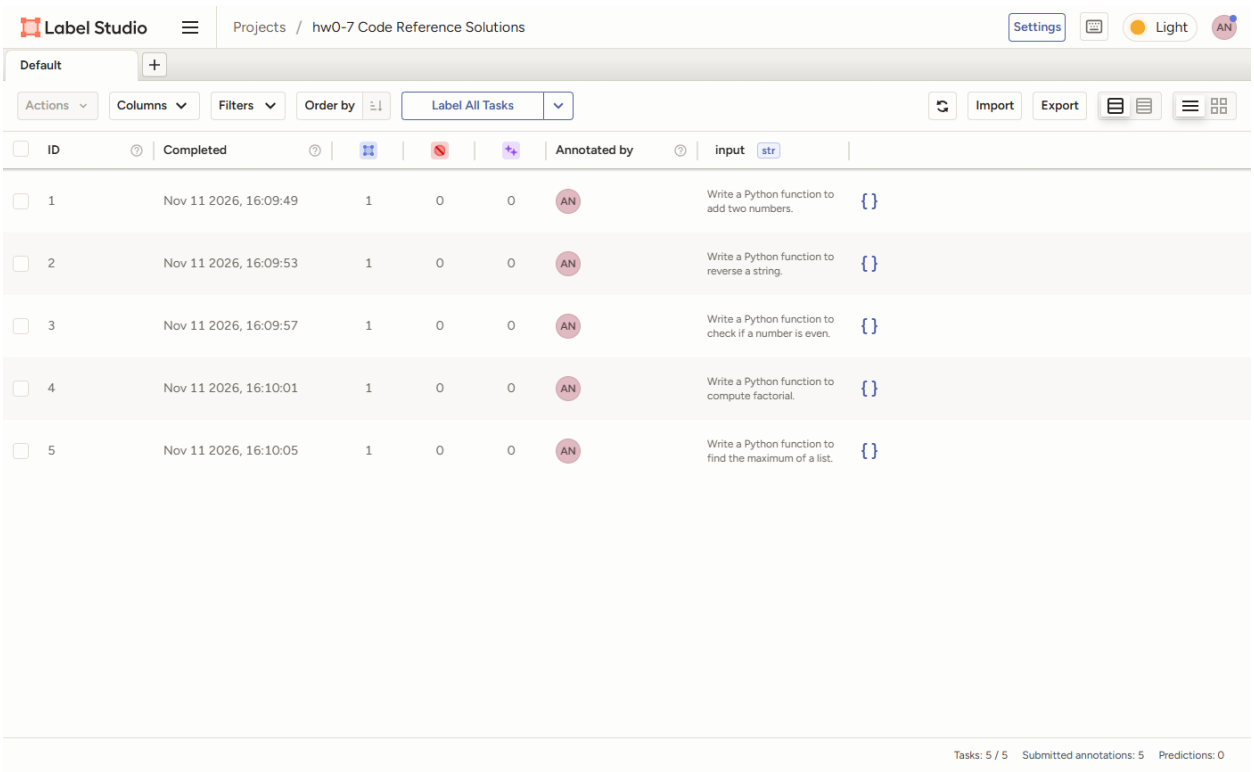


图 5 5 个任务全部完成标注

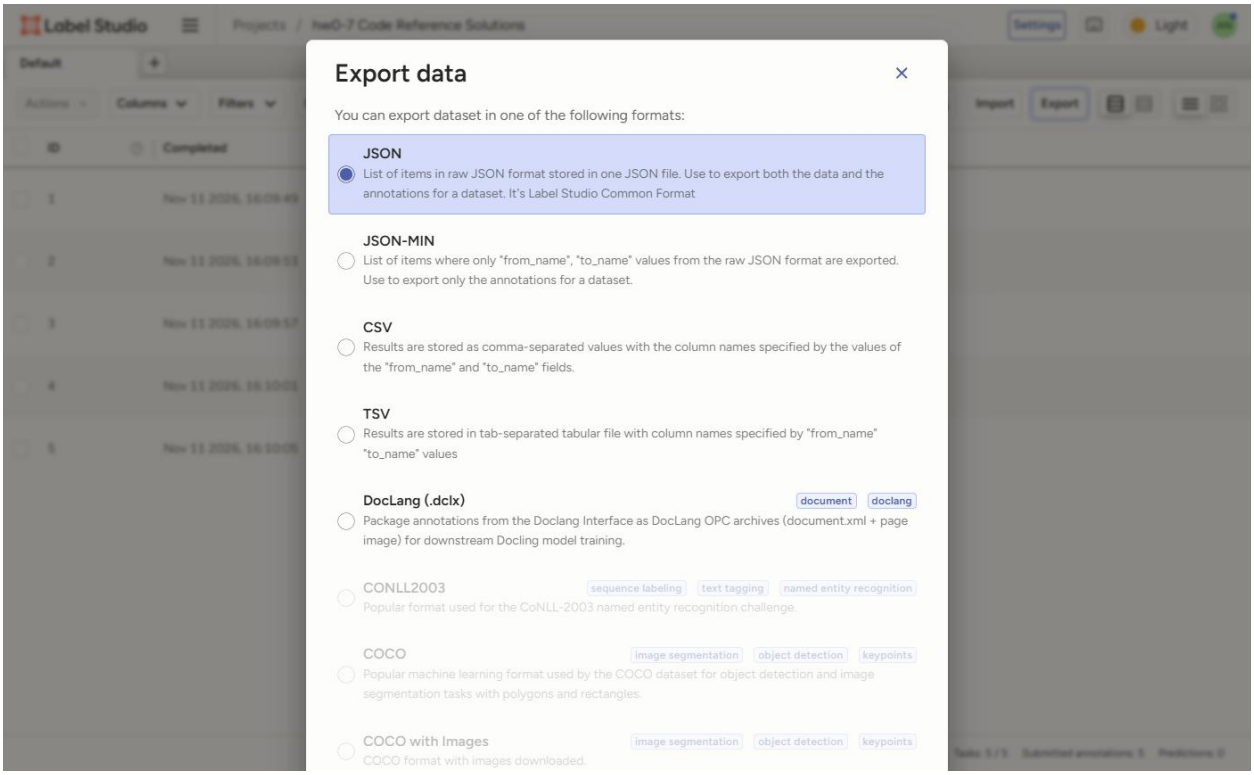


图 6 Export → JSON

参考答案由 Claude 编写，并在 Label Studio 界面中逐条填写提交（labelstudio/annotate_ui.py 驱动浏览器完成与人工相同的操作）。导出 JSON 中一条任务的结构（节选）：


```
{
  "id": 4,
  "data": {
    "input": "Write a Python function to compute factorial."
  },
  "annotations": [
    {
      "result": [
        {
          "value": {
            "choices": [
              "medium"
            ]
          },
          "id": "1PbdiWB6Ak",
          "from_name": "difficulty",
          "to_name": "input",
          "type": "choices",
          "origin": "manual"
        },
        {
          "value": {
            "text": [
              "def factorial(n):\n    if n < 0:\n        raise ValueError(\"factorial is not defined for negative
numbers\")\n    result = 1\n    for i in range(2, n + 1):\n        result *= i\n    return result\n"
            ]
          },
          "id": "HWYWXRX9aK",
          "from_name": "ideal",
```

```
          "to_name": "input",
          "type": "textarea",
          "origin": "manual"
        }
      ]
    }
  ]
}
```

3. 合并为 JSONL

python convert.py：读取导出 JSON 中每个任务最新一次未取消的标注，取出 ideal；按顺序与模型输出 CSV 的 sampled 合并。合并前检查：条数一致、导出的题目与 hw0-07_inputs.csv 一致、ideal 与 sampled 定义的函数名相同。结果 data/samples.jsonl：

```
{"input": "Write a Python function to add two numbers.", "ideal": "def add(a, b):\n    return a + b", "sampled": "def add(a, b): return a + b"}
{"input": "Write a Python function to reverse a string.", "ideal": "def reverse_string(s):\n    return s[::-1]", "sampled": "def reverse_string(s):\n    return ''.join(reversed(s))"}
{"input": "Write a Python function to check if a number is even.", "ideal": "def is_even(n):\n    return n % 2 == 0", "sampled": "def is_even(n):\n    if n % 2 == 0:\n        return True\n    else:\n        return False"}
{"input": "Write a Python function to compute factorial.", "ideal": "def factorial(n):\n    if n < 0:\n        raise ValueError(\"factorial is not defined for negative numbers\")\n    result = 1\n    for i in range(2, n + 1):\n        result *= i\n    return result", "sampled": "def factorial(n):\n    result = 1\n    for i in range(1, n):\n        result *= i\n    return result"}
{"input": "Write a Python function to find the maximum of a list.", "ideal": "def find_max(nums):\n    if not nums:\n        raise ValueError(\"find_max() of an empty list\")\n    largest = nums[0]\n    for x in nums[1:]:\n        if x > largest:\n            largest = x\n    return largest", "sampled": "def find_max(nums):\n    return sorted(nums)[0]"}
```

4. 评分提示词与 Eval 配置

4.1 评分提示词（modelgraded spec）

要点：角色设定；按行为而非与参考答案的相似度评分；要求在典型输入和题目隐含的边界情况上手动追踪代码；不因题目未要求的输入校验、类型标注扣分；5 档 rubric；两档之间犹豫时取低分。eval_type 为 cot_classify，Evals 会在提示词后要求模型先逐步推理，最后单独一行输出 1–5。input_outputs: {input: sampled} 表示样本中已有输出，Evals 不再生成，直接评分。

```
code_grading:
  prompt: |-
    You are a senior Python engineer grading solutions to short programming exercises. Your grades are used to measure how well a model writes code, so be consistent and strict about correctness.

    Task: decide how correct the submitted solution is, using the reference solution as one example of a correct answer.

    [BEGIN DATA]
    ************
    [Task]: {input}
    ************
    [Reference solution]:
    {ideal}
    ************
```

```
    [Submitted solution]:
    {sampled}
    ************
    [END DATA]

    How to judge:
    - Judge behaviour, not similarity. A submission that uses a different approach from the reference but returns
the right result for every valid input is fully correct.
    - Trace the submitted code on a few typical inputs and on the edge cases the task implies (for example 0 or 1
for factorial, an empty or one-character string, negative numbers). Compare the results with what the task asks
for.
    - Only consider what the task asks for. Do not lower the grade for missing input validation, type hints or
docstrings that the task did not request.

    Scoring rubric:
    5 - fully correct: correct result for all valid inputs, including the implied edge cases; clear code.
    4 - correct, minor issues: correct results, but with a minor weakness such as needless verbosity, avoidable
inefficiency or unidiomatic style.
    3 - partially correct: correct for typical inputs but wrong for an implied edge case, or only part of the
task is done.
    2 - major issues: the approach is related to the task, but a bug (for example an off-by-one error) makes most
results wrong.
    1 - incorrect: does not solve the task: computes something else, cannot run, or is missing.

    If you are torn between two grades, choose the lower one.
  choice_strings: "12345"
  choice_scores:
    "1": 1
    "2": 2
    "3": 3
    "4": 4
    "5": 5
  input_outputs:
    input: sampled
```

4.2 Eval 注册

```
code-grading:
  id: code-grading.dev.v0
  description: Grade model-written Python functions from 1 to 5 against a human reference solution, with an LLM
judge.
  metrics: [score]

code-grading.dev.v0:
  class: evals.elsuite.modelgraded.classify:ModelBasedClassify
  args:
    samples_jsonl: code_grading/samples.jsonl
    modelgraded_spec: code_grading
    eval_type: cot_classify
    eval_kwargs:
      temperature: 0
      max_tokens: 1024

# Same eval with the judge sampled at temperature 1, to measure how much the judge's own randomness moves the
scores.
code-grading-t1:
  id: code-grading-t1.dev.v0
  description: code-grading with the judge at temperature 1 (judge-variance study).
  metrics: [score]

code-grading-t1.dev.v0:
  class: evals.elsuite.modelgraded.classify:ModelBasedClassify
  args:
```

```
    samples_jsonl: code_grading/samples.jsonl
    modelgraded_spec: code_grading
    eval_type: cot_classify
    eval_kwargs:
      temperature: 1
      max_tokens: 1024

# Evals only routes model names it already knows to the chat endpoint, so gpt-4.1 is registered explicitly.
gpt-4.1-chat:
  class: evals_ext:ChatCompletionFn
  args:
    model: gpt-4.1
```

4.3 运行

单次运行：PYTHONPATH=. oaieval gpt-4.1-chat code-grading --registry_path evals_registry --record_path results/run.jsonl。run_eval.py 把它重复 5 次（--seed 1…5），每次的记录文件保存在 results/runs/<eval>/run_N.jsonl。



图 7 oaieval 运行结果（Final report：score 为 5 题平均分）

为在 Windows 和新版 openai SDK 上运行做了三处兼容处理：gpt-4.1 不在 Evals 的已知模型列表中，会被发到旧的 completions 接口，因此注册为 chat 完成函数（evals_ext.ChatCompletionFn）；Evals 汇总 token 用量时无法处理新版 SDK 的嵌套 *_details 字段，替换为只累加整数字段；blobfile 不接受 E:\ 这类盘符路径，记录路径改用相对路径。均不影响评分。

5. 结果与解读

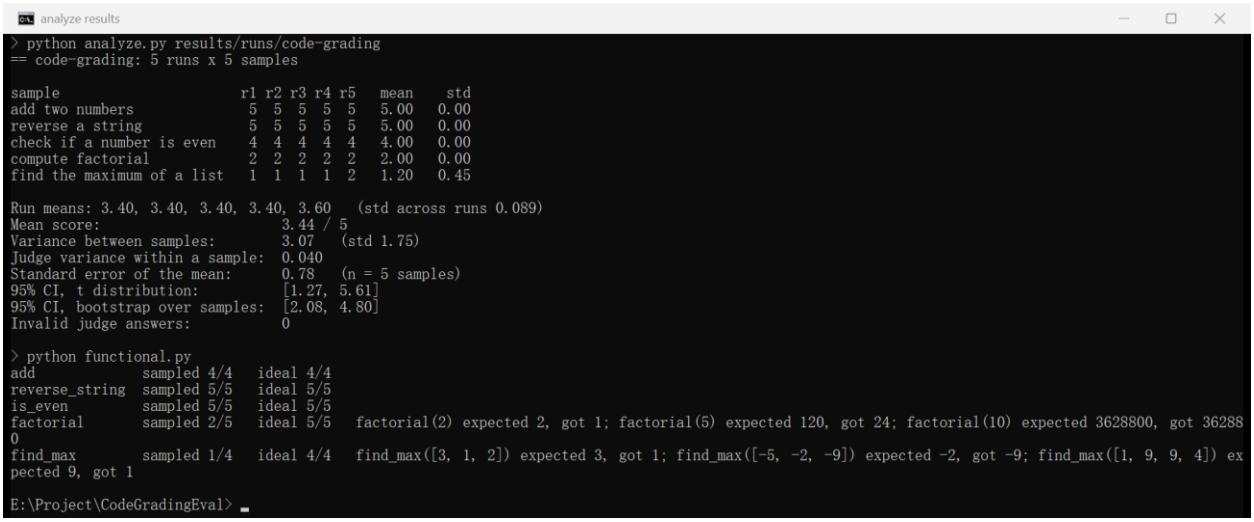


图 8 5 次运行的统计与单元测试对照

指标	评委 temperature 0	评委 temperature 1
平均分	3.44 / 5	3.48 / 5
题间方差（标准差）	3.07（1.75）	2.85（1.69）
同一题跨运行的评委方差	0.040	0.060
各次运行平均分	3.4, 3.4, 3.4, 3.4, 3.6	3.6, 3.4, 3.6, 3.4, 3.4
平均分的标准误（n = 5 题）	0.78	0.76
95% CI（t 分布）	[1.27, 5.00]（上限截到 5）	[1.38, 5.00]
95% CI（bootstrap）	[2.08, 4.80]	[2.16, 4.80]
无效回答	0	0

题目	评委分数 r1–r5	单元测试（sampled）	原因
add	5 5 5 5 5	4/4	正确
reverse_string	5 5 5 5 5	5/5	正确，写法与参考答案不同
is_even	4 4 4 4 4	5/5	结果正确，但 if/else 包装布尔值，略冗长
factorial	2 2 2 2 2	2/5	range(1, n) 少乘 n，只有 0! 和 1! 正确
find_max	1 1 1 1 2	1/4	sorted(nums)[0] 返回最小值

• 平均分 3.44 掩盖了两极分布：3 个正确（4–5 分、测试全过），2 个错误（1–2 分）。题间方差 3.07 很大，平均分本身不代表任何一题。

• 不确定性主要来自题目太少，而不是评委。重复评分 5 次，平均分只在 3.4–3.6 之间波动（标准差 0.09），评委方差只有 0.04；但只有 5 道题，平均分的标准误 0.78，95% 置信区间宽约 3 分。要缩小区间，需要更多题目，多跑几次没用。

• temperature 0 也不完全确定：find_max 有 1 次得 2 分；截图那次额外运行中 is_even 得了 5 分。两处都在 rubric 的边界上：sorted(nums)[0] 只差一个下标（应为 [-1]），评委有时认为是「思路相关但有 bug」（2），有时是「算的是别的东西」（1）；if/else 是否算「小问题」也是主观判断。改进办法：在 rubric 里给边界情况举例，或多次评分取中位数。

• 评委与规则检查一致：测试全过的得 4–5 分，大多数测试失败的得 1–2 分。按单元测试的通过率是 3/5。

5.1 评委推理示例（find_max，第 1 次运行）

```
ness for typical input**:
   - For `nums = [1, 2, 3]`, `sorted(nums)` is `[1, 2, 3]`, so `sorted(nums)[0]` is `1`. The maximum should be `3`. This
is incorrect.
   - For `nums = [5, 2, 9, 1]`, `sorted(nums)` is `[1, 2, 5, 9]`, so `sorted(nums)[0]` is `1`. The maximum should be `9`.
This is incorrect.

2. **Correctness for edge cases**:
   - For a single-element list, e.g., `[7]`, `sorted(nums)[0]` is `7`, which is correct.
   - For an empty list, `sorted([])[0]` will raise an `IndexError`, not a `ValueError` as in the reference, but the main
issue is with the main logic.

3. **What does the code actually do?**
   - It returns the minimum element of the list, not the maximum.

4. **Does it solve the task?**
   - No, it does not find the maximum of the list. It finds the minimum.

5. **Is the approach related to the task?**
   - The function is related to the task in that it processes the list and returns a value from it, but it returns the
wrong value.

6. **Does it work for any valid input?**
   - No, it gives the wrong result for all lists with more than one element.

7. **Does it run?**
   - Yes, it runs for non-empty lists, but gives the wrong result.

Conclusion:
- The code does not solve the task; it computes something else (the minimum).
- According to the rubric, this is a "1 - incorrect: does not solve the task: computes something else, cannot run, or is
missing."

1

1
```

6. 自动化测试

```
test_pipeline.py::test_real_export_converts_to_three_key_jsonl  PASSED
test_pipeline.py::test_multiline_csv_cells_are_read_whole  PASSED
test_pipeline.py::test_latest_annotation_wins_and_cancelled_ignored  PASSED
test_pipeline.py::test_tasks_are_ordered_by_id  PASSED
test_pipeline.py::test_merge_errors[no-annotation]  PASSED
```

```
test_pipeline.py::test_merge_errors[empty-ideal]  PASSED
test_pipeline.py::test_merge_errors[count-mismatch]  PASSED
test_pipeline.py::test_merge_errors[wrong-tasks]  PASSED
test_pipeline.py::test_merge_errors[misaligned]  PASSED
test_pipeline.py::test_functional_check_matches_known_bugs  PASSED
test_pipeline.py::test_functional_check_handles_broken_code  PASSED
test_pipeline.py::test_read_record  PASSED
test_pipeline.py::test_summary_statistics  PASSED
test_pipeline.py::test_invalid_answers_count_as_lowest_grade  PASSED
test_pipeline.py::test_t95_table  PASSED
test_pipeline.py::test_eval_config  PASSED
test_pipeline.py::test_usage_patch_ignores_nested_details  PASSED
============================ 17 passed in 4.04s ==============================
```

7. 代码

convert.py

```
"""Merge the Label Studio export (input + ideal) with the model outputs (sampled) into an eval JSONL.

    python convert.py
    python convert.py --export labelstudio/export.json --outputs data/hw0-07_inputs_model_output.csv
"""
from __future__ import annotations

import argparse
import csv
import json
import re
import shutil
import sys
from pathlib import Path

ROOT = Path(__file__).parent
EXPORT = ROOT / "labelstudio" / "export.json"
INPUTS = ROOT / "data" / "hw0-07_inputs.csv"
OUTPUTS = ROOT / "data" / "hw0-07_inputs_model_output.csv"
JSONL = ROOT / "data" / "samples.jsonl"
REGISTRY_COPY = ROOT / "evals_registry" / "data" / "code_grading" / "samples.jsonl"


class ConvertError(ValueError):
    pass


def read_column(path: Path, column: str) -> list[str]:
    with open(path, encoding="utf-8-sig", newline="") as f:
        rows = list(csv.DictReader(f))
    if rows and column not in rows[0]:
        raise ConvertError(f"{path.name}: no column {column!r} (found {list(rows[0])})")
    return [r[column].strip() for r in rows]


def labelled(export: list[dict]) -> list[dict]:
    """One {input, ideal, difficulty} per task, in task order, from each task's latest submitted annotation."""
    out = []
    for task in sorted(export, key=lambda t: t["id"]):
        anns = [a for a in task.get("annotations", []) if not a.get("was_cancelled")]
        if not anns:
            raise ConvertError(f"task {task['id']} ({task['data'].get('input')!r}) has no annotation")
        ann = max(anns, key=lambda a: a.get("updated_at") or a.get("created_at") or "")
        fields = {r["from_name"]: r["value"] for r in ann["result"]}
        text = fields.get("ideal", {}).get("text", [])
        ideal = "\n".join(text).strip("\n")
```

```
        if not ideal.strip():
            raise ConvertError(f"task {task['id']} has an empty ideal")
        out.append({"input": task["data"]["input"].strip(), "ideal": ideal,
                    "difficulty": (fields.get("difficulty", {}).get("choices") or [None])[0]})
    return out


def function_name(code: str) -> str | None:
    m = re.search(r"^\s*def\s+(\w+)\s*\(", code, re.M)
    return m.group(1) if m else None


def merge(labels: list[dict], sampled: list[str], inputs: list[str] | None = None) -> list[dict]:
    if len(labels) != len(sampled):
        raise ConvertError(f"{len(labels)} labelled tasks but {len(sampled)} model outputs")
    if inputs is not None and [l["input"] for l in labels] != inputs:
        raise ConvertError("exported tasks are not the tasks in hw0-07_inputs.csv, in the same order")
    rows = []
    for i, (label, out) in enumerate(zip(labels, sampled), 1):
        ref, got = function_name(label["ideal"]), function_name(out)
        if ref and got and ref != got:
            raise ConvertError(f"row {i}: ideal defines {ref}() but sampled defines {got}(); files are misaligned")
        rows.append({"input": label["input"], "ideal": label["ideal"], "sampled": out})
    return rows


def write_jsonl(rows: list[dict], path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8", newline="\n") as f:
        for row in rows:
            f.write(json.dumps(row, ensure_ascii=False) + "\n")


def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--export", type=Path, default=EXPORT)
    ap.add_argument("--inputs", type=Path, default=INPUTS)
    ap.add_argument("--outputs", type=Path, default=OUTPUTS)
    ap.add_argument("--out", type=Path, default=JSONL)
    args = ap.parse_args(argv)
    try:
        labels = labelled(json.loads(args.export.read_text(encoding="utf-8")))
        rows = merge(labels, read_column(args.outputs, "sampled"), read_column(args.inputs, "input"))
    except ConvertError as err:
        print(f"error: {err}", file=sys.stderr)
        return 1
    write_jsonl(rows, args.out)
    REGISTRY_COPY.parent.mkdir(parents=True, exist_ok=True)
    shutil.copyfile(args.out, REGISTRY_COPY)
    print(f"wrote {len(rows)} samples to {args.out} (copied to {REGISTRY_COPY.relative_to(ROOT)})")
    for label in labels:
        print(f"  [{label['difficulty']}] {label['input']}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

run_eval.py

```
"""Run the OpenAI Evals model-graded eval several times and keep every record file.

    python run_eval.py                    # code-grading (judge at temperature 0), 5 runs
    python run_eval.py --eval code-grading-t1 --runs 5
"""
from __future__ import annotations
```

```
import argparse
import os
import shutil
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).parent
REGISTRY = ROOT / "evals_registry"


def oaieval() -> str:
    exe = shutil.which("oaieval", path=str(Path(sys.executable).parent)) or shutil.which("oaieval")
    if not exe:
        raise SystemExit("oaieval not found: pip install evals")
    return exe


def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--eval", default="code-grading")
    ap.add_argument("--judge", default="gpt-4.1-chat", help="completion function used as the judge")
    ap.add_argument("--runs", type=int, default=5)
    args = ap.parse_args(argv)
    if not os.environ.get("OPENAI_API_KEY"):
        raise SystemExit("set OPENAI_API_KEY first")

    out = ROOT / "results" / "runs" / args.eval
    out.mkdir(parents=True, exist_ok=True)
    for i in range(1, args.runs + 1):
        record = out / f"run_{i}.jsonl"
        record.unlink(missing_ok=True)
        # blobfile (used by Evals to write records) rejects Windows drive paths, so pass paths relative to ROOT.
        cmd = [oaieval(), args.judge, args.eval, "--registry_path", REGISTRY.relative_to(ROOT).as_posix(),
               "--record_path", record.relative_to(ROOT).as_posix(), "--seed", str(i)]
        print(f"[{i}/{args.runs}] " + " ".join(Path(c).name if c == cmd[0] else c for c in cmd), flush=True)
        env = {**os.environ, "PYTHONPATH": str(ROOT)}  # so the registry can import evals_ext
        proc = subprocess.run(cmd, cwd=ROOT, env=env, capture_output=True, text=True, encoding="utf-8", errors="replace")
        (out / f"run_{i}.log").write_text(proc.stdout + proc.stderr, encoding="utf-8")
        if proc.returncode != 0:
            print(proc.stdout[-2000:] + proc.stderr[-4000:], file=sys.stderr)
            return proc.returncode
        report = [l for l in (proc.stdout + proc.stderr).splitlines() if "Final report" in l or
l.strip().startswith(("score", "counts/"))]
        print("    " + " | ".join(l.split("]")[-1].strip() for l in report[-8:]), flush=True)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

analyze.py

```
"""Summarise OpenAI Evals record files: mean score, variance and uncertainty.

    python analyze.py                       # every eval under results/runs/
    python analyze.py results/runs/code-grading
"""
from __future__ import annotations

import json
import random
import statistics as st
import sys
from dataclasses import dataclass
from pathlib import Path
```

```
ROOT = Path(__file__).parent
# Two-sided 95% critical values of Student's t, by degrees of freedom.
T95 = {1: 12.706, 2: 4.303, 3: 3.182, 4: 2.776, 5: 2.571, 6: 2.447, 7: 2.365, 8: 2.306, 9: 2.262, 10: 2.228,
       15: 2.131, 20: 2.086, 30: 2.042}


def t95(df: int) -> float:
    return T95.get(df) or T95[max(k for k in T95 if k <= df)] if df >= 1 else float("nan")


@dataclass
class Judgement:
    index: int          # position of the sample in samples.jsonl
    choice: str         # "1".."5" or "__invalid__"
    score: float | None
    reasoning: str      # the judge's full answer


def read_record(path: Path) -> list[Judgement]:
    """Parse one oaieval record file. Sample ids look like '<eval>.<split>.<version>.<index>'."""
    metrics, answers = {}, {}
    for line in path.read_text(encoding="utf-8").splitlines():
        if not line.strip():
            continue
        event = json.loads(line)
        sid = event.get("sample_id")
        if sid is None:
            continue
        idx = int(sid.rsplit(".", 1)[1])
        if event.get("type") == "metrics":
            metrics[idx] = event["data"]
        elif event.get("type") == "sampling":
            sampled = event["data"].get("sampled")
            answers[idx] = sampled[0] if isinstance(sampled, list) else str(sampled)
    return [Judgement(i, str(m["choice"]), m.get("score"), answers.get(i, "")) for i, m in sorted(metrics.items())]


@dataclass
class Summary:
    runs: int
    samples: int
    run_means: list[float]
    per_sample: list[list[float]]   # per_sample[i] = scores of sample i across runs
    invalid: int

    @property
    def sample_means(self) -> list[float]:
        return [st.mean(s) for s in self.per_sample]

    @property
    def mean(self) -> float:
        return st.mean(self.sample_means)

    @property
    def between_variance(self) -> float:
        """Variance of the per-sample mean scores: how much the test cases differ from each other."""
        return st.variance(self.sample_means) if self.samples > 1 else 0.0

    @property
    def judge_variance(self) -> float:
        """Average within-sample variance across runs: the judge's own randomness."""
        return st.mean(st.variance(s) for s in self.per_sample) if self.runs > 1 else 0.0

    @property
    def stderr(self) -> float:
        return (self.between_variance / self.samples) ** 0.5
```

```
    @property
    def t_interval(self) -> tuple[float, float]:
        half = t95(self.samples - 1) * self.stderr
        return self.mean - half, self.mean + half

    def bootstrap_interval(self, reps: int = 10000, seed: int = 0) -> tuple[float, float]:
        rng = random.Random(seed)
        means = self.sample_means
        boots = sorted(st.mean(rng.choices(means, k=len(means))) for _ in range(reps))
        return boots[int(0.025 * reps)], boots[int(0.975 * reps) - 1]


def summarise(runs: list[list[Judgement]]) -> Summary:
    n = len(runs[0])
    if any(len(r) != n for r in runs):
        raise ValueError("runs graded different numbers of samples")
    invalid = sum(j.score is None for r in runs for j in r)
    # An invalid answer scores as 1 (the lowest grade), so a broken judge can't inflate the mean.
    per_sample = [[float(r[i].score) if r[i].score is not None else 1.0 for r in runs] for i in range(n)]
    run_means = [st.mean(float(j.score) if j.score is not None else 1.0 for j in r) for r in runs]
    return Summary(len(runs), n, run_means, per_sample, invalid)


def load_runs(folder: Path) -> list[list[Judgement]]:
    files = sorted(folder.glob("run_*.jsonl"), key=lambda p: int(p.stem.split("_")[1]))
    return [read_record(p) for p in files]


def report(name: str, s: Summary, labels: list[str] | None = None) -> str:
    labels = labels or [f"sample {i}" for i in range(s.samples)]
    width = max(len(l) for l in labels)
    lines = [f"== {name}: {s.runs} runs x {s.samples} samples", "",
             f"{'sample':<{width}}  " + " ".join(f"r{i + 1}" for i in range(s.runs)) + "   mean    std"]
    for label, scores in zip(labels, s.per_sample):
        sd = st.stdev(scores) if len(scores) > 1 else 0.0
        lines.append(f"{label:<{width}}  " + " ".join(f"{x:>2.0f}" for x in scores) + f"   {st.mean(scores):.2f}
{sd:.2f}")
    lo, hi = s.t_interval
    blo, bhi = s.bootstrap_interval()
    lines += [
        "",
        "Run means: " + ", ".join(f"{m:.2f}" for m in s.run_means)
        + (f"   (std across runs {st.stdev(s.run_means):.3f})" if s.runs > 1 else ""),
        f"Mean score:                      {s.mean:.2f} / 5",
        f"Variance between samples:        {s.between_variance:.2f}   (std {s.between_variance ** 0.5:.2f})",
        f"Judge variance within a sample:  {s.judge_variance:.3f}",
        f"Standard error of the mean:      {s.stderr:.2f}   (n = {s.samples} samples)",
        f"95% CI, t distribution:          [{lo:.2f}, {hi:.2f}]",
        f"95% CI, bootstrap over samples:  [{blo:.2f}, {bhi:.2f}]",
        f"Invalid judge answers:           {s.invalid}",
    ]
    return "\n".join(lines)


def main(argv=None) -> int:
    argv = sys.argv[1:] if argv is None else argv
    folders = [Path(a) for a in argv] or sorted(p for p in (ROOT / "results" / "runs").iterdir() if p.is_dir())
    samples = [json.loads(l) for l in (ROOT / "data" / "samples.jsonl").read_text(encoding="utf-8").splitlines() if l]
    labels = [s["input"].replace("Write a Python function to ", "").rstrip(".") for s in samples]
    texts = []
    for folder in folders:
        runs = load_runs(folder)
        if not runs:
            continue
        s = summarise(runs)
        texts.append(report(folder.name, s, labels))
        summary = {"eval": folder.name, "runs": s.runs, "samples": s.samples, "mean": s.mean,
```

```
                    "between_variance": s.between_variance, "judge_variance": s.judge_variance,
                    "stderr": s.stderr, "t_interval": s.t_interval, "bootstrap_interval": s.bootstrap_interval(),
                    "run_means": s.run_means, "per_sample": s.per_sample, "invalid": s.invalid}
        (folder / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    text = "\n\n".join(texts)
    print(text)
    (ROOT / "results" / "analysis.txt").write_text(text + "\n", encoding="utf-8")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

functional.py

```
"""Rule-based cross-check: execute each solution against unit tests, so the judge's grades can be compared with
behaviour."""
from __future__ import annotations

import json
import multiprocessing as mp
from dataclasses import dataclass
from pathlib import Path

ROOT = Path(__file__).parent

# Test cases by function name: (args, expected). They cover typical inputs and the edge cases each task implies.
TESTS: dict[str, list[tuple[tuple, object]]] = {
    "add": [((2, 3), 5), ((-1, 1), 0), ((0, 0), 0), ((0.5, 0.25), 0.75)],
    "reverse_string": [(("abc",), "cba"), (("",), ""), (("a",), "a"), (("racecar",), "racecar"), (("ab cd",), "dc ba")],
    "is_even": [((4,), True), ((7,), False), ((0,), True), ((-2,), True), ((-3,), False)],
    "factorial": [((0,), 1), ((1,), 1), ((2,), 2), ((5,), 120), ((10,), 3628800)],
    "find_max": [(([3, 1, 2],), 3), (([-5, -2, -9],), -2), (([7],), 7), (([1, 9, 9, 4],), 9)],
}


@dataclass
class Result:
    function: str | None
    passed: int
    total: int
    failures: list[str]

    @property
    def rate(self) -> float:
        return self.passed / self.total if self.total else 0.0


def _run(code: str, out) -> None:
    namespace: dict = {}
    try:
        exec(compile(code, "<solution>", "exec"), namespace)
    except Exception as err:  # noqa: BLE001 - any failure to load the code is a result
        out.put(("load", repr(err)))
        return
    funcs = [k for k, v in namespace.items() if k in TESTS and callable(v)]
    if not funcs:
        out.put(("load", "no known function defined"))
        return
    name = funcs[0]
    results = []
    for args, expected in TESTS[name]:
        try:
            got = namespace[name](*args)
            results.append((", ".join(map(repr, args)), repr(expected), repr(got), got == expected))
        except Exception as err:  # noqa: BLE001
            results.append((", ".join(map(repr, args)), repr(expected), repr(err), False))
```

```
    out.put((name, results))


def check(code: str, timeout: float = 5.0) -> Result:
    """Run `code` in a separate process (so a hang or crash cannot take the caller down) and score it."""
    queue = mp.Queue()
    proc = mp.Process(target=_run, args=(code, queue))
    proc.start()
    proc.join(timeout)
    if proc.is_alive():
        proc.terminate()
        return Result(None, 0, 1, ["timed out"])
    name, payload = queue.get()
    if name == "load":
        return Result(None, 0, 1, [payload])
    failures = [f"{name}({a}) expected {e}, got {g}"
                for a, e, g, ok in payload if not ok]
    return Result(name, sum(ok for *_, ok in payload), len(payload), failures)


def check_file(path: Path) -> list[dict]:
    rows = [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]
    out = []
    for row in rows:
        s, i = check(row["sampled"]), check(row["ideal"])
        out.append({"input": row["input"], "function": s.function or i.function,
                    "sampled_passed": s.passed, "sampled_total": s.total, "sampled_failures": s.failures,
                    "ideal_passed": i.passed, "ideal_total": i.total})
    return out


if __name__ == "__main__":
    for r in check_file(ROOT / "data" / "samples.jsonl"):
        print(f"{r['function']:<15} sampled {r['sampled_passed']}/{r['sampled_total']}   "
              f"ideal {r['ideal_passed']}/{r['ideal_total']}   {'; '.join(r['sampled_failures'])}")
```

evals_ext.py

```
"""Completion function for OpenAI Evals that sends any model name to the chat endpoint.

Evals sends model names it does not know (such as gpt-4.1) to the legacy completions endpoint, and the built-in
OpenAIChatCompletionFn cannot be used from the registry because the registry passes it a `registry` argument.
"""
from evals.completion_fns.openai import OpenAIChatCompletionFn


class ChatCompletionFn(OpenAIChatCompletionFn):
    def __init__(self, model: str, registry=None, **kwargs):
        super().__init__(model=model, **kwargs)


def _patch_token_usage() -> None:
    """oaieval sums every usage field, but newer openai SDKs include nested *_details objects, which crash the sum.

    Replace the summary with one that only adds the integer counters. Grading results are unaffected.
    """
    import evals.cli.oaieval as cli

    def add_token_usage_to_result(result, recorder):
        totals: dict[str, int] = {}
        for event in recorder.get_events("sampling"):
            for key, value in dict(event.data.get("usage") or {}).items():
                if isinstance(value, int):
                    totals[key] = totals.get(key, 0) + value
        for key, value in totals.items():
            result[f"usage_{key}"] = value
```

```
    cli.add_token_usage_to_result = add_token_usage_to_result


_patch_token_usage()
```

labelstudio/setup_project.py

```
"""Create the Label Studio project, import the tasks, and export the annotations, via the REST API.

    python labelstudio/setup_project.py create     # project + labeling interface + import hw0-07_inputs.csv
    python labelstudio/setup_project.py export     # write labelstudio/export.json

Reads LS_URL (default http://localhost:8080) and LS_TOKEN from the environment or labelstudio/.env.local.
"""
from __future__ import annotations

import json
import os
import sys
from pathlib import Path

import requests

HERE = Path(__file__).parent
ROOT = HERE.parent
TITLE = "hw0-7 Code Reference Solutions"


def settings() -> tuple[str, dict]:
    env = {}
    local = HERE / ".env.local"
    if local.exists():
        env = dict(line.split("=", 1) for line in local.read_text().split() if "=" in line)
    url = os.environ.get("LS_URL", env.get("LS_URL", "http://localhost:8080")).rstrip("/")
    token = os.environ.get("LS_TOKEN", env.get("LS_TOKEN"))
    if not token:
        raise SystemExit("set LS_TOKEN")
    return url, {"Authorization": f"Token {token}"}


def find_project(url, headers) -> dict | None:
    resp = requests.get(f"{url}/api/projects", headers=headers, params={"title": TITLE}, timeout=30)
    resp.raise_for_status()
    body = resp.json()
    projects = body["results"] if isinstance(body, dict) else body
    return next((p for p in projects if p["title"] == TITLE), None)


def create() -> int:
    url, headers = settings()
    if (project := find_project(url, headers)) is not None:
        print(f"project {project['id']} already exists: {url}/projects/{project['id']}")
        return 0
    config = (HERE / "label_config.xml").read_text(encoding="utf-8")
    resp = requests.post(f"{url}/api/projects", headers=headers, timeout=30, json={
        "title": TITLE,
        "description": "Write a correct reference solution (ideal) for each programming task.",
        "label_config": config,
        "show_skip_button": False,
    })
    resp.raise_for_status()
    project = resp.json()
    with open(ROOT / "data" / "hw0-07_inputs.csv", "rb") as f:
        imp = requests.post(f"{url}/api/projects/{project['id']}/import", headers=headers, timeout=60,
                            files={"file": ("hw0-07_inputs.csv", f, "text/csv")})
    imp.raise_for_status()
    print(f"created project {project['id']} with {imp.json().get('task_count')} tasks: {url}/projects/{project['id']}")
```

```
    return 0


def export(out: Path = HERE / "export.json") -> int:
    url, headers = settings()
    project = find_project(url, headers)
    if project is None:
        raise SystemExit("project not found; run create first")
    resp = requests.get(f"{url}/api/projects/{project['id']}/export", headers=headers, timeout=60,
                        params={"exportType": "JSON", "download_all_tasks": "true"})
    resp.raise_for_status()
    tasks = resp.json()
    out.write_text(json.dumps(tasks, indent=2, ensure_ascii=False), encoding="utf-8")
    done = sum(bool(t.get("annotations")) for t in tasks)
    print(f"exported {len(tasks)} tasks ({done} annotated) to {out}")
    return 0 if done == len(tasks) else 1


if __name__ == "__main__":
    action = sys.argv[1] if len(sys.argv) > 1 else "create"
    sys.exit({"create": create, "export": export}[action]())
```

labelstudio/annotate_ui.py

```
"""Enter the reference solutions through the Label Studio web UI (the same steps an annotator takes), with screenshots.

    python labelstudio/annotate_ui.py

It drives Microsoft Edge with Playwright: log in, open the labeling stream, and for each task type the
reference solution into the `ideal` text area, choose a difficulty and press Submit.
"""
from __future__ import annotations

import os
import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

ROOT = Path(__file__).parent.parent
SHOTS = ROOT / "docs"
URL = os.environ.get("LS_URL", "http://127.0.0.1:8080")
EMAIL = os.environ.get("LS_EMAIL", "annotator@example.com")
PASSWORD = os.environ.get("LS_PASSWORD", "hw07-annotator")

# Reference solutions, written for this homework (task text -> (ideal, difficulty)).
IDEALS = {
    "Write a Python function to add two numbers.": (
        "def add(a, b):\n    return a + b", "easy"),
    "Write a Python function to reverse a string.": (
        "def reverse_string(s):\n    return s[::-1]", "easy"),
    "Write a Python function to check if a number is even.": (
        "def is_even(n):\n    return n % 2 == 0", "easy"),
    "Write a Python function to compute factorial.": (
        "def factorial(n):\n"
        "    if n < 0:\n"
        "        raise ValueError(\"factorial is not defined for negative numbers\")\n"
        "    result = 1\n"
        "    for i in range(2, n + 1):\n"
        "        result *= i\n"
        "    return result", "medium"),
    "Write a Python function to find the maximum of a list.": (
        "def find_max(nums):\n"
        "    if not nums:\n"
        "        raise ValueError(\"find_max() of an empty list\")\n"
        "    largest = nums[0]\n"
        "    for x in nums[1:]:\n"
```

```
        "        if x > largest:\n"
        "            largest = x\n"
        "    return largest", "easy"),
}


def main() -> int:
    SHOTS.mkdir(exist_ok=True)
    with sync_playwright() as p:
        browser = p.chromium.launch(channel="msedge", args=["--no-proxy-server"])
        page = browser.new_page(viewport={"width": 1400, "height": 860})
        page.goto(f"{URL}/user/login/", wait_until="domcontentloaded")
        page.fill("input[name=email]", EMAIL)
        page.fill("input[name=password]", PASSWORD)
        page.click("button[type=submit]")
        page.wait_for_url(lambda u: "login" not in u, timeout=30000)

        page.goto(f"{URL}/projects/1/data", wait_until="domcontentloaded")
        page.wait_for_timeout(3000)
        page.screenshot(path=str(SHOTS / "1-tasks-imported.png"))

        page.goto(f"{URL}/projects/1/data?labeling=1", wait_until="domcontentloaded")
        done = 0
        for _ in range(len(IDEALS) + 1):
            try:
                page.wait_for_selector("textarea", timeout=15000)
            except Exception:  # noqa: BLE001 - no more tasks in the stream
                break
            page.wait_for_timeout(800)
            task = next((t for t in IDEALS if page.get_by_text(t, exact=True).count()), None)
            if task is None:
                break
            ideal, difficulty = IDEALS[task]
            box = page.locator("textarea").first
            box.click()
            box.fill(ideal)
            box.press("End")
            page.locator(".lsf-choices").get_by_text(difficulty).first.click()
            page.wait_for_timeout(600)
            if done == 0:
                page.screenshot(path=str(SHOTS / "2-labeling.png"))
            page.get_by_test_id("bottombar-submit-button").click()
            page.wait_for_timeout(2500)
            done += 1
            print(f"submitted: {task} [{difficulty}]")

        page.goto(f"{URL}/projects/1/data", wait_until="domcontentloaded")
        page.wait_for_timeout(3000)
        page.screenshot(path=str(SHOTS / "3-all-labelled.png"))
        browser.close()
    print(f"{done} tasks submitted")
    return 0 if done == len(IDEALS) else 1


if __name__ == "__main__":
    sys.exit(main())
```